

Lab 3: Comprehensive Technical Documentation & Evidence Package

DevOps Lab 3: Ansible Configuration Automation & Terraform Infrastructure as Code (IaC)

1. Executive Summary

This consolidated document contains the complete technical documentation, source configuration files, terminal commands, execution verification, and visual evidence for both Lab 3 tasks: Question 1 (Ansible Automation) and Question 2 (Terraform IaC).

Evaluation Section	Implemented Tooling	Execution Result	Idempotency / Scaling
Q1: Configuration Automation	Ansible Core 2.21.4 + community.docker.docker	10/10 tasks OK, 7 changed	100% Idempotent (changed=0)
Q2: Infrastructure as Code	Terraform v1.13.3 + kreuzwerker/docker	4 resources added, state tracked	Scaled 2 → 4 Replicas (HTTP 200)

2. Complete Evidence Screenshots

Figure 1: Docker Host Environment & Target Node Status

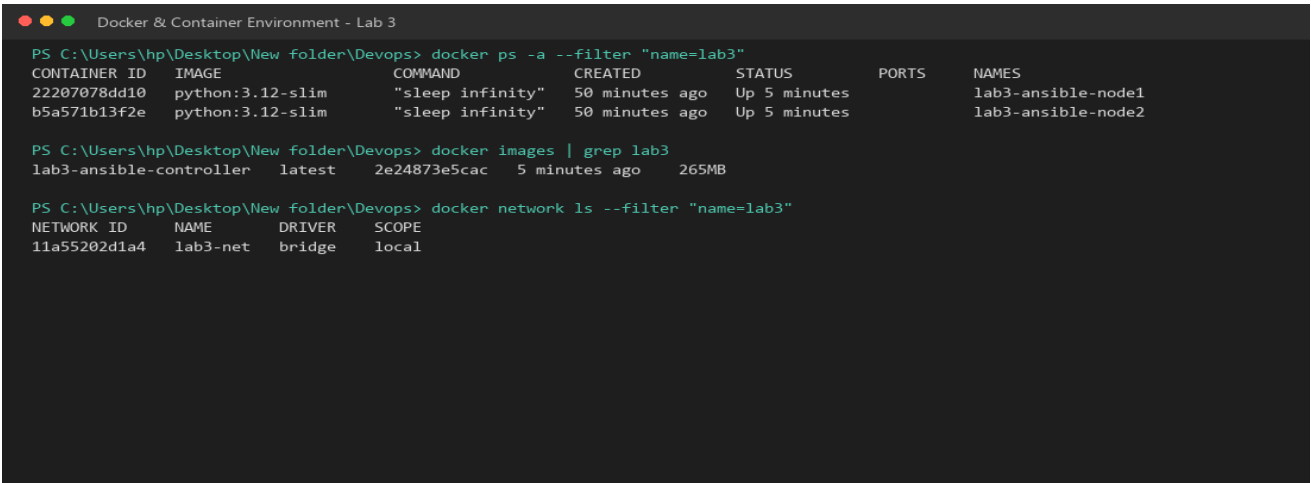


Figure 2: Ansible Inventory & Ad-Hoc Target Node Ping

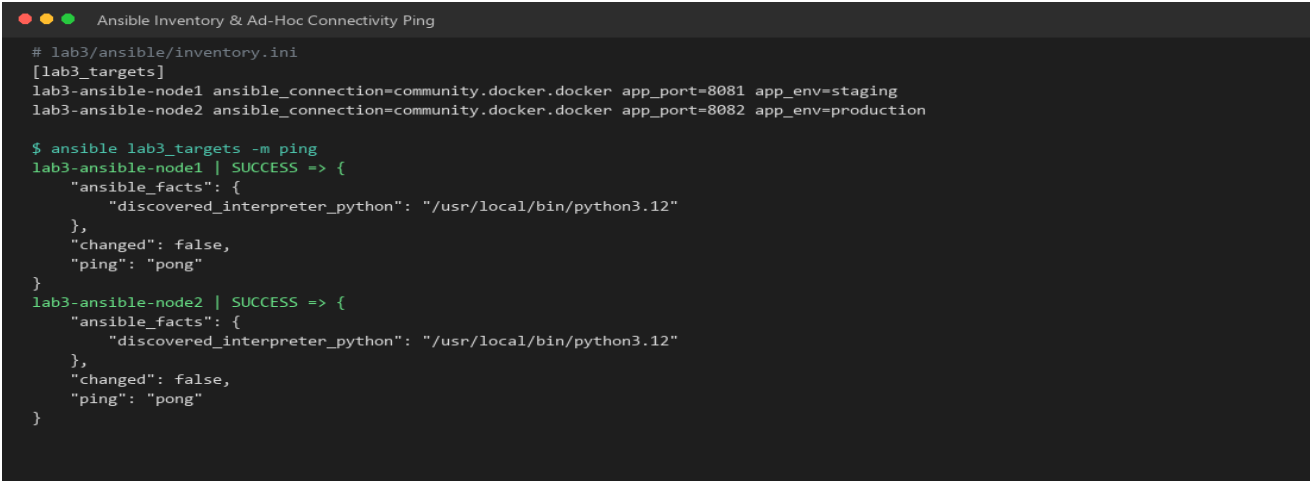


Figure 3: Ansible Master Playbook & Role Directory Structure

```
Ansible Site Playbook & Application Role Structure

# lab3/ansible/site.yml
---
- name: Configure Lab 3 application target nodes
  hosts: lab3_targets
  gather_facts: true
  roles:
    - role: app_config
  post_tasks:
    - name: Verify the application health endpoint responds on the node
      ansible.builtin.command:
        cmd: python /opt/lab3app/bin/healthcheck.py
        changed_when: false
        tags: [verify]

# Directory tree: lab3/ansible/roles/app_config
# |--- defaults/main.yml
# |--- files/ (app.py, healthcheck.py, ctl.sh)
# |--- handlers/main.yml
# |--- tasks/main.yml
# |--- templates/app.conf.j2
```

Figure 4: Target Node Inspection & Python 3.12 Runtime

```
Ansible Target Node Inspection

PS C:\Users\hp\Desktop\New folder\Devops> docker ps --filter "name=lab3-ansible-node"
CONTAINER ID   IMAGE          COMMAND                  STATUS        PORTS          NAMES
22207078dd10   python:3.12-slim "sleep infinity"        Up 8 minutes   lab3-ansible-node1
b5a571b13f2e   python:3.12-slim "sleep infinity"        Up 8 minutes   lab3-ansible-node2

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 python --version
Python 3.12.14

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 python --version
Python 3.12.14
```

Figure 5: Ansible Playbook Syntax Check

```
Ansible Syntax Check

$ ansible-playbook --syntax-check site.yml

playbook: site.yml

# Syntax check complete: No errors detected in playbook or role syntax.
```

Figure 6: Initial Playbook Execution (7 Changes Applied)

```
Ansible Playbook Initial Execution (Changes Applied)
$ ansible-playbook site.yml
PLAY [Configure Lab 3 application target nodes] *****

TASK [Gathering Facts] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Create lab3app system group] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Create lab3app service user] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Create application directory layout] *****
changed: [lab3-ansible-node1] => (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)
changed: [lab3-ansible-node2] => (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)

TASK [app_config : Deploy application configuration (template)] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Deploy application code] *****
changed: [lab3-ansible-node1] => (item=app.py, healthcheck.py, ctl.sh)
changed: [lab3-ansible-node2] => (item=app.py, healthcheck.py, ctl.sh)

TASK [app_config : Start (or restart) the lab3app service] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [app_config : Wait for the health endpoint to come up] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

RUNNING HANDLER [app_config : Restart lab3app] *****
changed: [lab3-ansible-node1]
changed: [lab3-ansible-node2]

TASK [Verify the application health endpoint responds on the node] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

PLAY RECAP *****
lab3-ansible-node1 : ok=10 changed=7 unreachable=0 failed=0
lab3-ansible-node2 : ok=10 changed=7 unreachable=0 failed=0
```

Figure 7: Second Playbook Execution (100% Idempotency, changed=0)

```
Ansible Playbook 2nd Run — Verification of Idempotency
$ ansible-playbook site.yml
PLAY [Configure Lab 3 application target nodes] *****

TASK [Gathering Facts] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Create lab3app system group] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Create lab3app service user] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Create application directory layout] *****
ok: [lab3-ansible-node1] => (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)
ok: [lab3-ansible-node2] => (item=/opt/lab3app/bin, /etc/lab3app, /var/log/lab3app, /var/run/lab3app)

TASK [app_config : Deploy application configuration (template)] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Deploy application code] *****
ok: [lab3-ansible-node1] => (item=app.py, healthcheck.py, ctl.sh)
ok: [lab3-ansible-node2] => (item=app.py, healthcheck.py, ctl.sh)

TASK [app_config : Start (or restart) the lab3app service] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [app_config : Wait for the health endpoint to come up] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

TASK [Verify the application health endpoint responds on the node] *****
ok: [lab3-ansible-node1]
ok: [lab3-ansible-node2]

PLAY RECAP *****
lab3-ansible-node1 : ok=9 changed=0 unreachable=0 failed=0
lab3-ansible-node2 : ok=9 changed=0 unreachable=0 failed=0
```

Figure 8: Direct Node Verification (Config, Process Status, Healthcheck)

```
Direct Verification on Ansible Target Nodes

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 cat /etc/lab3app/app.conf
# Lab 3 staging configuration — managed by Ansible, DO NOT EDIT
app_name = lab3app
environment = staging
listen_port = 8081
log_level = info
greeting = Hello from Lab 3 (Ansible-configured)
node = lab3-ansible-node1

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 /opt/lab3app/bin/ctl.sh status
lab3app running (pid 380)

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node1 python /opt/lab3app/bin/healthcheck.py
OK lab3app on port 8081 (env=staging)

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 cat /etc/lab3app/app.conf
# Lab 3 production configuration — managed by Ansible, DO NOT EDIT
app_name = lab3app
environment = production
listen_port = 8082
log_level = info
greeting = Hello from Lab 3 (Ansible-configured)
node = lab3-ansible-node2

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 /opt/lab3app/bin/ctl.sh status
lab3app running (pid 379)

PS C:\Users\hp\Desktop\New folder\Devops> docker exec lab3-ansible-node2 python /opt/lab3app/bin/healthcheck.py
OK lab3app on port 8082 (env=production)
```

Figure 9: Terraform IaC Configuration Files

```
Terraform IaC Configuration Files

# Directory: lab3/terraform
main.tf      - Defines docker provider, docker_network, docker_image, and docker_container (count)
variables.tf - Defines docker_host, network_name, image_name, replica_count, ports, environment
outputs.tf   - Outputs network ID/name, replica_count, and container instance metadata list
terraform.tfvars - Parameter values (replica_count = 2, base_host_port = 8181, env = production)

# Key resource definition in main.tf:
resource "docker_container" "app_service" {
  count = var.replica_count
  name  = "${var.container_name_prefix}-${count.index + 1}"
  image = docker_image.app_image.image_id
  command = ["python", "-m", "http.server", "8080"]
  networks_advanced { name = docker_network.app_network.name }
  ports { internal = 8080; external = var.base_host_port + count.index }
}
```

Figure 10: Terraform Initialization & Docker Provider Setup

```
Terraform Initialization (terraform init)

PS C:\Users\hp\Desktop\New folder\Devops\lab3\terraform> terraform init

Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0.2"...
- Installing kreuzwerker/docker v3.0.2...
- Installed kreuzwerker/docker v3.0.2 (self-signed, key ID BD080C4571C6104C)

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure.
```

Figure 11: Terraform Configuration Validation

```
Terraform Configuration Validation (terraform validate)

PS C:\Users\hp\Desktop\New folder\Devops\lab3\terraform> terraform validate

Success! The configuration is valid.
```

Figure 12: Initial Execution Plan (2 Replicas)

```
Terraform Execution Plan (replica_count = 2)

PS C:\Users\hp\Desktop\New folder\Devops\lab3\terraform> terraform plan

Terraform will perform the following actions:

# docker_network.app_network will be created
+ resource "docker_network" "app_network" { ... name = "lab3-tf-net" }

# docker_image.app_image will be created
+ resource "docker_image" "app_image" { ... name = "python:3.12-slim" }

# docker_container.app_service[0] will be created
+ resource "docker_container" "app_service" {
  + name = "lab3-tf-app-1"
  + ports { external = 8181; internal = 8080 }
}

# docker_container.app_service[1] will be created
+ resource "docker_container" "app_service" {
  + name = "lab3-tf-app-2"
  + ports { external = 8182; internal = 8080 }
}

Plan: 4 to add, 0 to change, 0 to destroy.
```

Figure 13: Initial Apply (2 Replicas Provisioned)

```
Terraform Apply (Initial 2 Replicas)

PS C:\Users\hp\Desktop\New folder\Devops\lab3\terraform> terraform apply -auto-approve

docker_network.app_network: Creating...
docker_image.app_image: Creating...
docker_image.app_image: Creation complete after 0s [id=sha256:f77ac9e44ae9...]
docker_network.app_network: Creation complete after 2s [id=4f44ce49361c...]
docker_container.app_service[0]: Creating...
docker_container.app_service[1]: Creating...
docker_container.app_service[0]: Creation complete after 0s [id=5df778fc8fde...]
docker_container.app_service[1]: Creation complete after 1s [id=81e8c2089be2...]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
container_instances = [
  { host_port = 8181, id = "5df778fc8fde...", ip_address = "172.22.0.2", name = "lab3-tf-app-1" },
  { host_port = 8182, id = "81e8c2089be2...", ip_address = "172.22.0.3", name = "lab3-tf-app-2" }
]
network_id = "4f44ce49361c2de675b4e59b3158dd7cdefab985f3eafaeddc6f6bc393df47c"
network_name = "lab3-tf-net"
replica_count = 2
```

Figure 14: Verification of 2 Active Provisioned Containers & Network

```
Provisioned Infrastructure Verification (2 Replicas)

PS C:\Users\hp\Desktop\New folder\Devops> docker ps --filter "name=lab3-tf"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
5df778fc8fde   f77ac9e44ae9   "python -m http.serv..." 8 seconds ago  Up 7 seconds  0.0.0.0:8181->8080/tcp             lab3-tf-app-1
81e8c2089be2   f77ac9e44ae9   "python -m http.serv..." 8 seconds ago  Up 7 seconds  0.0.0.0:8182->8080/tcp             lab3-tf-app-2

PS C:\Users\hp\Desktop\New folder\Devops> docker network inspect lab3-tf-net --format "{{.json .Containers}}"
{"5df778fc...":{"Name":"lab3-tf-app-1","IPv4Address":"172.22.0.2/16"},
"81e8c208...":{"Name":"lab3-tf-app-2","IPv4Address":"172.22.0.3/16"}}
```

Figure 15: Terraform Dynamic Scaling Plan (2 → 4 Replicas)

```
Terraform Scaling Plan (replica_count 2 -> 4)

# terraform.tfvars updated: replica_count = 4
PS C:\Users\hp\Desktop\New folder\Devops\lab3\terraform> terraform plan

Terraform will perform the following actions:

  # docker_container.app_service[2] will be created
  + resource "docker_container" "app_service" {
    + name     = "lab3-tf-app-3"
    + ports    = { external = 8183; internal = 8080 }
  }

  # docker_container.app_service[3] will be created
  + resource "docker_container" "app_service" {
    + name     = "lab3-tf-app-4"
    + ports    = { external = 8184; internal = 8080 }
  }

Plan: 4 to add, 0 to change, 2 to destroy.
Changes to Outputs:
  ~ replica_count = 2 -> 4
```

Figure 16: Scaling Apply Execution (4 Active Replicas)

```
Terraform Apply: Scaling 2 -> 4 Replicas

PS C:\Users\hp\Desktop\New folder\Devops\lab3\terraform> terraform apply -auto-approve

docker_container.app_service[3]: Creating...
docker_container.app_service[2]: Creating...
docker_container.app_service[3]: Creation complete after 1s [id=13a1046cc09d...]
docker_container.app_service[2]: Creation complete after 1s [id=170c821bffa5...]
docker_container.app_service[0]: Creation complete after 1s [id=4668cb768ef8...]
docker_container.app_service[1]: Creation complete after 1s [id=f575154d4b0d...]

Apply complete! Resources: 4 added, 0 changed, 2 destroyed.

Outputs:
container_instances = [
  { host_port = 8181, id = "4668cb76...", ip_address = "172.22.0.4", name = "lab3-tf-app-1" },
  { host_port = 8182, id = "f575154d...", ip_address = "172.22.0.5", name = "lab3-tf-app-2" },
  { host_port = 8183, id = "170c821b...", ip_address = "172.22.0.3", name = "lab3-tf-app-3" },
  { host_port = 8184, id = "13a1046c...", ip_address = "172.22.0.2", name = "lab3-tf-app-4" }
]
replica_count = 4
```

Figure 17: Verification of 4 Active Scaled Replicas & HTTP 200 Responses

```
Scaled Infrastructure Verification (4 Active Replicas)

PS C:\Users\hp\Desktop\New folder\Devops> docker ps --filter "name=lab3-tf"
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4668cb768ef8   f77ac9e44ae9   "python -m http.serv..." 5 seconds ago  Up 5 seconds  0.0.0.0:8181->8080/tcp             lab3-tf-app-1
f575154d4b0d   f77ac9e44ae9   "python -m http.serv..." 5 seconds ago  Up 5 seconds  0.0.0.0:8182->8080/tcp             lab3-tf-app-2
170c821bffa5   f77ac9e44ae9   "python -m http.serv..." 6 seconds ago  Up 6 seconds  0.0.0.0:8183->8080/tcp             lab3-tf-app-3
13a1046cc09d   f77ac9e44ae9   "python -m http.serv..." 6 seconds ago  Up 6 seconds  0.0.0.0:8184->8080/tcp             lab3-tf-app-4

PS C:\Users\hp\Desktop\New folder\Devops> Invoke-WebRequest -Uri "http://127.0.0.1:8181" -> StatusCode: 200 OK
PS C:\Users\hp\Desktop\New folder\Devops> Invoke-WebRequest -Uri "http://127.0.0.1:8182" -> StatusCode: 200 OK
PS C:\Users\hp\Desktop\New folder\Devops> Invoke-WebRequest -Uri "http://127.0.0.1:8183" -> StatusCode: 200 OK
PS C:\Users\hp\Desktop\New folder\Devops> Invoke-WebRequest -Uri "http://127.0.0.1:8184" -> StatusCode: 200 OK
```